

# String Templates

Core idioms first, then problems grouped by pattern.

For sliding window string problems (#3, #76, #567, #424), see `sliding_window.md` .

## Quick Reference Table

| #    | Name                          | Pattern            | Key technique                            | Time / Space            |
|------|-------------------------------|--------------------|------------------------------------------|-------------------------|
| 242  | Valid Anagram                 | Char count         | count++ then --                          | $O(n)$ / $O(1)$         |
| 383  | Ransom Note                   | Char count         | fill from magazine, drain from note      | $O(n)$ / $O(1)$         |
| 387  | First Unique Character        | Char count         | scan for count == 1                      | $O(n)$ / $O(1)$         |
| 49   | Group Anagrams                | Freq hash key      | 26-bucket key or sorted key              | $O(n \cdot k)$ / $O(n)$ |
| 451  | Sort Characters by Frequency  | Bucket sort        | count[], bucket by freq, iterate desc    | $O(n)$ / $O(n)$         |
| 5    | Longest Palindromic Substring | Expand from center | expand(i,i) odd + expand(i,i+1) even     | $O(n^2)$ / $O(1)$       |
| 647  | Palindromic Substrings        | Expand from center | same expand, count instead of max length | $O(n^2)$ / $O(1)$       |
| 8    | String to Integer (atoi)      | Parsing            | skip space → sign → digits → overflow    | $O(n)$ / $O(1)$         |
| 14   | Longest Common Prefix         | Vertical scan      | shrink prefix until all strings match    | $O(n \cdot k)$ / $O(1)$ |
| 443  | String Compression            | Write pointer      | read run, write char+count in-place      | $O(n)$ / $O(1)$         |
| 1047 | Remove Adjacent Duplicates    | Stack              | push; pop if top == current              | $O(n)$ / $O(n)$         |

## Core Idioms

```
// MENTAL MODEL: a lowercase string is a length-26 frequency vector; most string problems are vector ops on it.
// WHEN: "anagram", "char count", "group by letters", "first unique", "sort by frequency".

// 1. CHAR COUNT – lowercase letters
int[] count = new int[26];
// WHY ch - 'a': ASCII 'a'=97, so 'a'→0, 'b'→1, ..., 'z'→25 – a clean 0–25 bucket index into count[].
for (char ch : s.toCharArray()) count[ch - 'a']++;    // ← ch - 'a' maps a→0, b→1, ..., z→25

// 2. CHAR COUNT – digits 0–9
int[] count = new int[10];
for (char ch : s.toCharArray()) count[ch - '0']++;    // ← ch - '0' maps '0'→0, '9'→9

// 3. CHAR TO INTEGER – build number digit by digit
int num = 0;
for (int i = 0; i < s.length(); i++)
    num = num * 10 + (s.charAt(i) - '0');    // ← shift left × 10, add new digit

// 4. ANAGRAM HASH KEY – frequency-based (bucket sort style) O(k) vs sort O(k log k)
// WHY it works: anagrams share identical letter frequencies, so encoding the frequency vector
// ITSELF gives a unique key for the whole group – O(k) to build vs O(k log k) to sort the chars.
String hashKey(String s) {
    int[] count = new int[26];
    for (char ch : s.toCharArray()) count[ch - 'a']++;
    StringBuilder builder = new StringBuilder();
    for (int i = 0; i < 26; i++) {
        builder.append((char)('a' + i));    // ← letter as label
        builder.append(count[i]);    // ← its count
    }
    return builder.toString();    // e.g., "a2b0c1...z0"
}

// Alternative: sort the characters (simpler, slightly slower)
char[] chars = s.toCharArray();
Arrays.sort(chars);
String key = new String(chars);    // anagrams → same sorted key

// 5. BUCKET SORT – sort by frequency without comparison sort
int[] count = new int[128];    // ASCII
for (char ch : s.toCharArray()) count[ch]++;
List<Character>[] buckets = new List[s.length() + 1]; // index = frequency
for (int i = 0; i < 128; i++) {
    if (count[i] > 0) {
        int freq = count[i];
        if (buckets[freq] == null) buckets[freq] = new ArrayList<>();
        buckets[freq].add((char) i);
    }
}

StringBuilder builder = new StringBuilder();
for (int freq = buckets.length - 1; freq > 0; freq--) {    // ← descending frequency
    if (buckets[freq] == null) continue;
    for (char ch : buckets[freq])
        for (int i = 0; i < freq; i++) builder.append(ch);
}

// 6. EXPAND FROM CENTER – palindrome check in O(1) space
int expand(String s, int i, int j) {
    // i == j: odd length; i+1 == j: even length
    while (i >= 0 && j < s.length() && s.charAt(i) == s.charAt(j)) { i--; j++; }
    return j - i - 1;    // ← length of palindrome found
}
```

# Part 1 — Char Frequency

---

## #242 Valid Anagram

---

**Description:** Return true if `t` is an anagram of `s` (same characters, same counts).

**Intuition:** Anagrams have identical letter counts, so add up `s` and subtract `t` in one 26-slot array — all zeros means they match.

```
class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        int[] count = new int[26];
        for (char ch : s.toCharArray()) count[ch - 'a']++; // ← fill from s
        for (char ch : t.toCharArray()) count[ch - 'a']--; // ← drain from t
        for (int c : count) if (c != 0) return false;      // ← any non-zero → not anagram
        return true;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

---

## #383 Ransom Note

---

**Description:** Can `ransomNote` be constructed from the letters in `magazine` (each letter used at most once)?

**Intuition:** Stock the letter counts from the magazine, then draw down for each note letter — if any count goes negative the magazine is short.

```
class Solution {
    public boolean canConstruct(String ransomNote, String magazine) {
        int[] count = new int[26];
        for (char ch : magazine.toCharArray()) count[ch - 'a']++; // ← fill from magazine
        for (char ch : ransomNote.toCharArray()) if (--count[ch - 'a'] < 0) return false; // ← drain
        return true;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

---

## #387 First Unique Character in a String

---

**Description:** Return the index of the first non-repeating character. Return -1 if none exists.

**Intuition:** One pass to count every letter, a second pass to return the first whose count is exactly 1.

```
class Solution {
    public int firstUniqChar(String s) {
        int[] count = new int[26];
        for (char ch : s.toCharArray()) count[ch - 'a']++;
        for (int i = 0; i < s.length(); i++)
            if (count[s.charAt(i) - 'a'] == 1) return i; // ← first with count exactly 1
        return -1;
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

---

## Part 2 — Anagram Hashing & Bucket Sort

### #49 Group Anagrams

**Description:** Group strings that are anagrams of each other.

**Intuition:** Anagrams share identical letter frequencies, so the frequency vector encoded as a string is a unique group key —  $O(k)$  to build vs  $O(k \log k)$  to sort the characters.

**Key:** each anagram group shares the same frequency hash key. `computeIfAbsent` cleanly handles first-time group creation.

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strs) {
            String key = hashKey(s); // ← frequency-based key
            map.computeIfAbsent(key, k -> new ArrayList<>()).add(s); // ← group by key
        }
        return new ArrayList<>(map.values());
    }

    private String hashKey(String s) { //  $O(k)$  — no sort needed
        int[] count = new int[26];
        for (char ch : s.toCharArray()) count[ch - 'a']++; // ← count each char
        StringBuilder builder = new StringBuilder();
        for (int i = 0; i < 26; i++) {
            builder.append((char)('a' + i)); // ← letter label
            builder.append(count[i]); // ← its count
        }
        return builder.toString(); // e.g., "a1e1t1"
    }
}

// Time  $O(n \cdot k)$  | Space  $O(n)$  —  $k$  = avg string length
```

**Alternative key (sort-based, simpler but  $O(k \log k)$ ):**

```
char[] chars = s.toCharArray();
Arrays.sort(chars);
String key = new String(chars); // "eat" → "aet", "tea" → "aet"
```

### #451 Sort Characters By Frequency

**Description:** Sort characters in descending order of frequency. Ties can go in any order.

**Intuition:** Frequencies are bounded by string length, so bucket chars by their count and read buckets high-to-low — no  $O(n \log n)$  comparison sort needed.

**Key:** bucket sort by frequency — avoids  $O(n \log n)$  comparison sort. Each bucket holds all chars with that frequency.

```

class Solution {
    public String frequencySort(String s) {
        int[] count = new int[128];                // ASCII range
        for (char ch : s.toCharArray()) count[ch]++;

        List<Character>[] buckets = new List[s.length() + 1];    // ← bucket index = frequency
        for (int i = 0; i < 128; i++) {
            if (count[i] > 0) {
                int freq = count[i];
                if (buckets[freq] == null) buckets[freq] = new ArrayList<>();
                buckets[freq].add((char) i);
            }
        }

        StringBuilder builder = new StringBuilder();
        for (int freq = s.length(); freq > 0; freq--) {    // ← descending frequency
            if (buckets[freq] == null) continue;
            for (char ch : buckets[freq])
                for (int i = 0; i < freq; i++) builder.append(ch);    // ← repeat char `freq` times
        }
        return builder.toString();
    }
}
// Time O(n) | Space O(n)

```

## Part 3 — Palindrome (Expand from Center)

### Template

```

// Call for each center: odd-length palindromes (i == i) and even-length (i, i+1)
for (int i = 0; i < s.length(); i++) {
    // odd: single center at i
    // even: center between i and i+1
    processExpansion(s, i, i);    // odd
    processExpansion(s, i, i + 1); // even
}

// Shared expand helper – returns length of palindrome
int expand(String s, int i, int j) {
    while (i >= 0 && j < s.length() && s.charAt(i) == s.charAt(j)) { i--; j++; }
    return j - i - 1;
}

```

## #5 Longest Palindromic Substring

**Description:** Return the longest palindromic substring.

**Intuition:** Every palindrome has a center; try all  $2n-1$  centers (each char and each gap) and grow outward while characters mirror.

**Key:** expand from every center (odd and even), track the longest found.

```

class Solution {
    public String longestPalindrome(String s) {
        int n = s.length(), start = 0, maxLen = 1;
        for (int i = 0; i < n; i++) {
            int odd = expand(s, i, i);           // odd-length palindrome
            int even = expand(s, i, i + 1);      // even-length palindrome
            int len = Math.max(odd, even);
            if (len > maxLen) {
                maxLen = len;
                start = i - (len - 1) / 2;      // ← back-compute start from center
            }
        }
        return s.substring(start, start + maxLen);
    }
    private int expand(String s, int i, int j) {
        while (i >= 0 && j < s.length() && s.charAt(i) == s.charAt(j)) { i--; j++; }
        return j - i - 1;
    }
}

```

**Time**  $O(n^2)$  | **Space**  $O(1)$

## #647 Palindromic Substrings

**Description:** Count all palindromic substrings.

**Intuition:** Same center-expansion, but each successful step outward is itself one more palindrome, so accumulate a count rather than a max.

**Key:** same expand helper — count expansions instead of tracking max length.

```

class Solution {
    public int countSubstrings(String s) {
        int count = 0;
        for (int i = 0; i < s.length(); i++) {
            count += expandCount(s, i, i);      // odd: each expansion = 1 new palindrome
            count += expandCount(s, i, i + 1);  // even
        }
        return count;
    }
    private int expandCount(String s, int i, int j) {
        int count = 0;
        while (i >= 0 && j < s.length() && s.charAt(i) == s.charAt(j)) {
            count++;                            // ← each valid expansion = 1 palindrome
            i--; j++;
        }
        return count;
    }
}

```

**Time**  $O(n^2)$  | **Space**  $O(1)$

## #5 vs #647 — Same expand, different accumulation

|                 | #5 Longest Palindromic        | #647 Palindromic Substrings |
|-----------------|-------------------------------|-----------------------------|
| Per expansion:  | track max length              | count++                     |
| Outer loop:     | track best start+len          | accumulate count            |
| Return:         | s.substring(start, start+len) | count                       |
| Expand returns: | palindrome length             | palindrome count            |

## Part 4 — String Parsing & Building

### #8 String to Integer (atoi)

**Description:** Parse a string to an integer, handling leading spaces, optional sign, overflow, and non-digit stop.

**Intuition:** Process the string in strict phases — spaces, then sign, then digits accumulated as `num*10 + digit` — clamping to int bounds the moment overflow would occur.

**Key steps in order:** skip spaces → read sign → read digits ( `num = num * 10 + (ch - '0')` ) → clamp on overflow.

```
class Solution {
    public int myAtoi(String s) {
        int i = 0, n = s.length();
        while (i < n && s.charAt(i) == ' ') i++;           // ← 1. skip leading spaces
        if (i == n) return 0;
        int sign = 1;
        if (s.charAt(i) == '-') { sign = -1; i++; }        // ← 2. read sign
        else if (s.charAt(i) == '+') { i++; }
        long result = 0;
        while (i < n && Character.isDigit(s.charAt(i))) {
            result = result * 10 + (s.charAt(i++) - '0'); // ← 3. digit by digit
            if (result * sign > Integer.MAX_VALUE) return Integer.MAX_VALUE; // ← 4. clamp
            if (result * sign < Integer.MIN_VALUE) return Integer.MIN_VALUE;
        }
        return (int)(result * sign);
    }
}
```

**Time**  $O(n)$  | **Space**  $O(1)$

### #14 Longest Common Prefix

**Description:** Find the longest common prefix string among an array of strings.

**Intuition:** Start with the first string as the candidate prefix and trim its tail until every other string starts with it.

**Key:** use the first string as the candidate prefix. Shrink it until every string starts with it.

```
class Solution {
    public String longestCommonPrefix(String[] strs) {
        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (!strs[i].startsWith(prefix))           // ← shrink until prefix matches
                prefix = prefix.substring(0, prefix.length() - 1);
            if (prefix.isEmpty()) return "";
        }
        return prefix;
    }
}
```

**Time**  $O(n \cdot k)$  | **Space**  $O(1)$

### #443 String Compression

**Description:** Compress the char array in-place. Each group `aaa` becomes `a3`. Single chars stay as-is. Return new length.

**Intuition:** Two pointers — read scans each run while write lays down the compressed `char + count` behind it; write never overtakes read, so it's safe in place.

**Key:** write pointer `write` advances independently of read pointer `i`. Count run length, then write `char + (count if > 1)`.

```

class Solution {
    public int compress(char[] chars) {
        int write = 0, i = 0;
        while (i < chars.length) {
            char ch = chars[i];
            int count = 0;
            while (i < chars.length && chars[i] == ch) { i++; count++; } // ← count run
            chars[write++] = ch; // ← write char
            if (count > 1)
                for (char c : Integer.toString(count).toCharArray()) // ← write digits
                    chars[write++] = c;
        }
        return write;
    }
}

```

**Time**  $O(n)$  | **Space**  $O(1)$

## #1047 Remove All Adjacent Duplicates in String

**Description:** Repeatedly remove adjacent equal characters until no more exist.

**Intuition:** A stack mirrors the partially-cleaned string — if the next char equals the top it's an adjacent pair, so pop instead of pushing.

**Key:** stack — push each char; if it equals the top, they form a pair → pop instead. Stack holds the result after all removals.

```

class Solution {
    public String removeDuplicates(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char ch : s.toCharArray()) {
            if (!stack.isEmpty() && stack.peek() == ch) {
                stack.pop(); // ← pair → cancel
            } else {
                stack.push(ch);
            }
        }
        StringBuilder builder = new StringBuilder();
        while (!stack.isEmpty()) builder.append(stack.pop());
        return builder.reverse().toString();
    }
}

```

**Time**  $O(n)$  | **Space**  $O(n)$

# String Idioms Cheat Sheet

```
// Char ↔ index
'a' + i           // int → char (i=0→'a', i=25→'z')
ch - 'a'          // char → index (a→0, z→25)
ch - '0'          // char → digit (0→0, 9→9)
Character.isDigit(ch)
Character.isLetter(ch)
Character.toLowerCase(ch)

// String operations
s.charAt(i)                // char at index
s.toCharArray()           // → char[]
new String(charArray)      // char[] → String
String.valueOf(charArray)  // char[] → String
s.substring(i, j)          // [i, j)
s.contains(sub)            // boolean
s.startsWith(prefix)       // boolean
s.split("\\s+")            // split on whitespace

// Building strings
StringBuilder builder = new StringBuilder();
builder.append(ch);         builder.append(str);
builder.deleteCharAt(i);
builder.reverse();
builder.toString();
String.join(", ", list)     // join with delimiter

// computeIfAbsent – key pattern for grouping
map.computeIfAbsent(key, k -> new ArrayList<>()).add(value);
```

## Pattern Chooser

| Signal                               | Pattern                                                                      |
|--------------------------------------|------------------------------------------------------------------------------|
| Are two strings anagrams?            | Count array: fill from s, drain from t                                       |
| Group strings by anagram             | Frequency hash key ( <code>hashCode</code> ) or <code>Arrays.sort</code> key |
| Sort/rank by character frequency     | Bucket sort: <code>count[]</code> , then <code>buckets[freq]</code>          |
| Longest/count palindromic substrings | Expand from center (i,i) odd + (i,i+1) even                                  |
| Parse a number from string           | <code>num = num * 10 + (ch - '0')</code>                                     |
| Remove adjacent duplicate pairs      | Stack: push, pop on match                                                    |
| Sliding window over characters       | See <code>sliding_window.md</code>                                           |